Assignment #4 - Neural Networks

Context

This assignment is an opportunity to demonstrate your knowledge and practice solving problems about convolutional and recurrent neural networks. This assignment contains an implementation component (i.e. you will be asked to write code to solve one or more problems) and an experiment component (i.e. you will be asked to experiment with your code and report results).

Logistics

Assignment due date: 2026-03-20

Assignments are to be submitted electronically through Brightspace. It is your responsibility to ensure that your assignment is submitted properly and that all the files for the assignment are included. Copying of assignments is NOT allowed. High-level discussion of assignment work with others is acceptable, but each individual or small group is expected to do the work themselves.

Implementation Part

Programming language: Python 3

For all parts of the implementation, you may use the Python Standard Library (https://docs.python.org/3/library/) and the following packages (and any packages they depend on): NumPy, Pandas, PyTorch, scikit-learn, SciPy. Unless explicitly indicated below or explicitly approved by the instructor, you may not use any additional packages.

You must implement your code yourself; do not copy-and-paste code from other sources. Please ensure your implementation follows the specifications provided; it may be tested on several different test cases for correctness. Please make sure your code is readable; it may also be manually assessed for correctness. You do not need to prove correctness of your implementation.

You must submit your implementation as a single file named "assignment4.py" with functions as described below (you may have other variables/functions/classes in your file). Attached is skeleton code indicating the format your implementation should take.

Experiments Part

You must submit your report on experimental results as a single file named "assignment4.pdf".

Implementation Part

For the implementation, we will use the Linnaeus 5 dataset. This dataset consists of 32-by-32 pixel photographic images from five different classes: berry, bird, dog, flower, other. This dataset is publicly available here: https://chaladze.com/l5/ (we will use the 32x32 version). Note that the quality of the images within this has not been verified. You should ensure your implementation is robust to any quality issues with the dataset.

Each JPEG image in the images folder represents one 32x32 RGB image. The provided data is already arranged into a train folder and test folder. The data is also arranged into a folder for each class. For our purposes, we will not use information about the class each image is associated with.

Question 1 [10 marks]

Create a PyTorch dataset class for this dataset (i.e. a class that subclasses torch.utils.data.Dataset). The dataset should be usable for training or testing a neural network in PyTorch. The data generator should perform appropriate pre-processing on the images (e.g. scaling, normalization, etc.).

The data generator class must be called "Linnaes5Dataset", and it should have three member functions: "__init__", "__len__", "__getitem__" (you may also use other helper functions).

The function "__init__" is a constructor that should take one input argument (in addition to "self"). The first input argument is the full path to the directory containing the train (or test) set.

The function "__len__" should take zero input arguments (in addition to "self"). It should return the total number of samples in the dataset.

The function "__getitem__" should take one input argument (in addition to "self"). The first input argument is a sample index. The function should return two values: (1) a sample of data (i.e. one image) and (2) the same sample of data (i.e. the same image).

Question 2 [10 marks]

Write a function that creates, trains, and evaluates a convolutional neural network autoencoder to encode an image from the Linnaeus 5 dataset. The encoder component of the network should perform two-dimensional convolution over the input. The decoder component of the network should perform two-dimensional transposed convolution over the input.

The function must be called "linnaeus5_autoencoder".

The function should take one input argument: (1) the full file path to the directory containing the training dataset.

The function should return three values: (1) a PyTorch module object (subclassing "torch.nn.Module") that is a trained autoencoder for this dataset, (2) performance of the model on a subset of data used for training, and (3) the performance of the model on a subset of data used for validation.

(Hint: use the data generator you wrote; randomly split dataset into a training set and a validation set)

Experiments Part

Question 3 [10 marks]

Conduct a series of experiments on the autoencoder you implemented for this. For each of these experiments, plot the performance on the training set and validation set as a function of the parameters.

a) Experiment on the size of the encoding.
b) Experiment on the number of convolutional layers in the encoder and decoder.
c) Experiment on the number of epochs of training.

Report the parameters and performance of the model with the best performance. Comment on any trends you observed in the results of your experiments.

d) Using the best performing model you found, compute performance (i.e. reconstruction loss) on the held-out test set.

(Hint: use the data generator you wrote)

Question 4 [10 marks]

Conduct another series of experiments on the autoencoder you implemented for this. For each of these experiments, plot the performance on the training set and validation set as a function of the parameters.

a) Experiment on the use of pre-processing within the data generator.
b) Experiment on the use of batch normalization for each layer.
c) Experiment on the use of L1 or L2 regularization of the parameters.
d) Experiment on the use of dropout.
e) Experiment on the value of the learning rate.

Report the parameters and performance of the model with the best performance. Comment on any trends you observed in the results of your experiments.

f) Using the best performing model you found, compute performance (i.e. reconstruction loss) on the held-out test set.

(Hint: use the data generator you wrote)